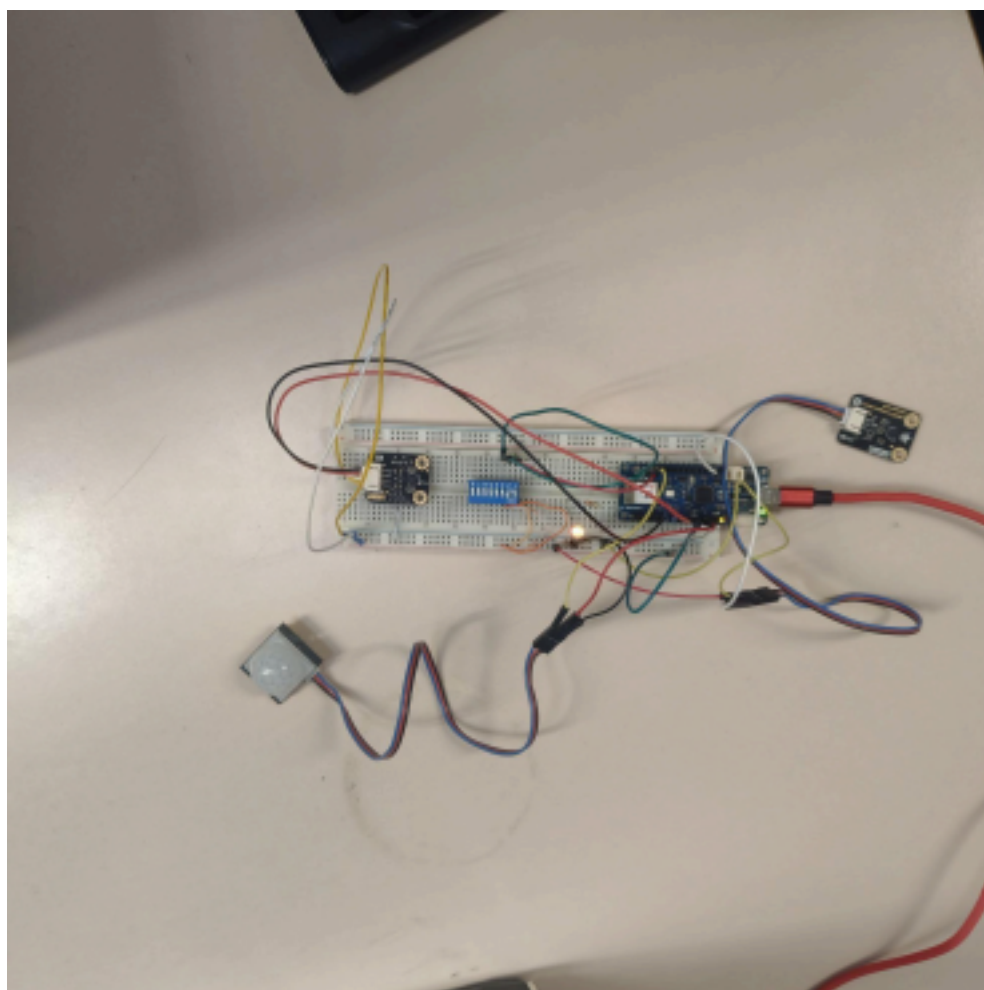




Arduino 2: Aggiunta sensore movimento

Questo è il nostro progetto

Componenti del gruppo	Incerti Matteo - Deovich Filippo
Classe	5DIT
Data inizio progetto	12/01/2026
Data fine progetto	26/01/2026

1

Introduzione

In questa esperienza l'obiettivo è ampliare il progetto realizzato in Arduino 1 integrando un sensore di movimento PIR e una IMU Gravity BMI160 a 6 assi. Il progetto prevede la gestione simultanea di più sensori, ciascuno attivabile o disattivabile tramite interruttori fisici, e l'organizzazione del codice tramite l'uso di branch Git per mantenere separato lo sviluppo delle nuove funzionalità. La libreria FastIMU viene utilizzata per comunicare con la BMI160, permettendo la lettura calibrata dei dati di accelerazione e giroscopio. Le misure dell'IMU e del sensore di temperatura LM35 vengono effettuate ogni 3 secondi, mentre il sensore PIR rileva il

movimento in tempo reale accendendo un LED di segnalazione.

Studio di fattibilità economica

Componente	Dettaglio	Quantità	Costo	Esempio
Arduino wifi mkr1010		1	40€	
Digital Infrared motion sensor SKU_SEN0018_		1	5€	
DFRobot LM35 Linear Temperature Sensor SKU_DFR0023_		1	5€	
Led rosso		1	0,20€	
DIP switch Interruttore		1	2€	
Breadboard		1	5€	
Gravity BMI160 6-Axis Inertial Motion Sensor SKU SEN0250 prezzo		1	16,92€	

2

Studio di fattibilità tecnica

Il progetto è tecnicamente fattibile perché la scheda Arduino utilizzata è in grado di gestire tutti i sensori richiesti senza difficoltà. Il sensore di temperatura LM35 utilizza un ingresso analogico, mentre il sensore di movimento PIR e il LED funzionano tramite pin digitali. La IMU Gravity BMI160 comunica tramite il bus I²C, che è già supportato nativamente da Arduino, quindi non sono necessari componenti aggiuntivi o circuiti complessi. Dal punto di vista software, la libreria FastIMU semplifica l'utilizzo della BMI160, permettendo di inizializzare il sensore, calibrarlo e leggere correttamente i dati di accelerazione e giroscopio. Le letture dei sensori vengono effettuate ogni 3 secondi, garantendo un funzionamento stabile e ordinato del programma. Inoltre, grazie agli interruttori, è possibile attivare o disattivare singolarmente il sensore di temperatura, il PIR e la IMU, rendendo il sistema flessibile e facile da controllare. Anche l'integrazione del sensore PIR risulta semplice, poiché fornisce un segnale digitale diretto che permette di rilevare il movimento e di accendere un LED come segnalazione visiva. Nel

complesso, l'hardware e il software scelti sono adeguati e consentono di realizzare il progetto in modo affidabile, rendendolo adatto a un'esperienza didattica e facilmente estendibile.

Diario di progetto

Data	Cose fatte	tempo	Cose da fare	tempo
12/01/2026	Abbiamo collegato i due account Sourcetree, abbiamo iniziato a fare il codice e siamo riusciti a collegare il sensore alla breadboard.	2:00h	Provare se il codice e se i collegamenti fatti funzionano.	

3

15/01/2026	Abbiamo provato il nostro codice, ma nel serial monitor ci da solo la temperatura quindi abbiamo smontato tutto e riprovato a farlo.	2:00h	continuare a montare i collegamenti e vedere se va il codice questa volta	
22/01/26	abbiamo corretto alcuni collegamenti a livello hardware e anche il codice non funziona come dovrebbe	2:00h	finire di fare il codice e correggere eventuali errori	

26/01/26	Abbiamo capito come mai non andava il codice e siamo così riusciti a far funzionare il tutto, e abbiamo cominciato a fare la relazione	2:00h	finire la relazione	
----------	--	-------	---------------------	--

Gli aspetti tecnologici

Il progetto utilizza diverse tecnologie hardware e software che lavorano insieme per permettere il corretto funzionamento del sistema. Al centro del progetto c'è Arduino, che svolge il ruolo di unità di controllo e si occupa della lettura dei sensori, dell'elaborazione dei dati e della comunicazione tramite porta seriale. Grazie alla sua semplicità e affidabilità, Arduino è adatto a gestire più dispositivi contemporaneamente in un contesto didattico come questo. Dal punto di vista **hardware**, vengono utilizzati sensori di tipo diverso, ognuno con una funzione specifica. Il sensore LM35 fornisce una misura analogica della temperatura, mentre il sensore PIR rileva il movimento attraverso un segnale digitale e permette di accendere un LED come segnalazione visiva. La IMU Gravity BMI160 rappresenta il componente tecnologicamente più avanzato, poiché integra un accelerometro e un giroscopio a sei assi e comunica con Arduino tramite il protocollo I²C, che consente lo scambio di dati in modo rapido ed efficiente.

Dal punto di vista **software**, la libreria FastIMU è fondamentale per la gestione della BMI160, poiché semplifica l'accesso ai dati del sensore e riduce la complessità del codice. Il programma è strutturato in modo da eseguire le letture dei sensori ogni 3 secondi e da permettere all'utente di attivare o disattivare le varie funzionalità tramite interruttori fisici. La comunicazione seriale viene utilizzata per visualizzare i dati raccolti, facilitando il controllo e il debug del sistema.

Codice:

```
#include "FastIMU.h"
#include <Wire.h>

#define IMU_ADDRESS 0x68

BMI160 IMU;

calData calib = {0};
AccelData accelData;
GyroData gyroData;
```

```

// PIN
const int pinLM35 = A0;
const int pinPIR = 2;
const int pinLED = 6;

const int pinSwitchTemp = 3;
const int pinSwitchPIR = 4;
const int pinSwitchIMU = 5;

// STATI
bool enableIMU = false;
bool enablePIR = false;

void setup() {
  Serial.begin(9600);
  Wire.begin();

  pinMode(pinPIR, INPUT); // PIR CORRETTO
  pinMode(pinLED, OUTPUT);

  pinMode(pinSwitchTemp, INPUT_PULLUP);
  pinMode(pinSwitchPIR, INPUT_PULLUP);
  pinMode(pinSwitchIMU, INPUT_PULLUP);

  IMU.init(calib, IMU_ADDRESS);

  Serial.println("SISTEMA AVVIATO - ATTENDI WARMUP PIR");
}

void loop() {

  // LETTURA INTERRUITORI
  bool readTemp = (digitalRead(pinSwitchTemp) == LOW);
  enablePIR = (digitalRead(pinSwitchPIR) == HIGH);
  enableIMU = (digitalRead(pinSwitchIMU) == LOW);

  // ----- LM35 -----
  if (readTemp) {
    int raw = analogRead(pinLM35);

    float voltage = raw * 3.3 / 1023.0;
    float temperature = voltage * 100.0;

    Serial.print("TEMPERATURA: ");
    Serial.print(temperature);
    Serial.println(" C");
  }

  // ----- PIR -----

```

```

if (enablePIR) {
  int motion = digitalRead(pinPIR);

  if (motion == HIGH) {
    digitalWrite(pinLED, HIGH);
    Serial.println("RILEVATO");
  } else {
    digitalWrite(pinLED, LOW);
    Serial.println("NESSUN MOVIMENTO");
  }
} else {
  digitalWrite(pinLED, LOW);
  Serial.println("PIR DISABILITATO");
}

// ----- BMI160 -----
if (enableIMU) {
  IMU.update();
  IMU.getAccel(&accelData);
  IMU.getGyro(&gyroData);

  Serial.println("BMI160 IMU:");
  Serial.print("ACCEL X: "); Serial.print(accelData.accelX);
  Serial.print(" Y: "); Serial.print(accelData.accelY);
  Serial.print(" Z: "); Serial.println(accelData.accelZ);

  Serial.print("GYRO X: "); Serial.print(gyroData.gyroX);
  Serial.print(" Y: "); Serial.print(gyroData.gyroY);
  Serial.print(" Z: "); Serial.println(gyroData.gyroZ);
}

Serial.println("-----");

delay(200); // piccolo delay per rendere il PIR reattivo
}

```

Link

https://wiki.dfrobot.com/DFRobot_LM35_Linear_Temperature_Sensor_SKU_DFR0023

https://wiki.dfrobot.com/Digital_Infrared_motion_sensor_SKU_SEN0018

https://wiki.dfrobot.com/Gravity_BMI160_6-Axis_Inertial_Motion_Sensor_SKU_SEN02

<https://chatgpt.com/>